NSS COLLEGE OF ENGINEERING, PALAKKAD

(Government Aided)

Department of Computer Science and Engineering

Project Report

on

BUFFER OVERFLOW AND CONTROL HAZARDS IN RISC-V ARCHITECTURE

Submitted in fulfillment of the requirements for the course
Computer Organization & Architecture (PBCST404)

Submitted by:

Adharsh K (Register No: NSS24CS010)
Angelino Saul (Register No: NSS24CS036)
Aswin Manoj (Register No: NSS24CS054)
Aswin R (Register No: NSS24CS055)

Group No: 4

Academic Year: 2025-2026

NSS COLLEGE OF ENGINEERING, PALAKKAD

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **"Buffer Overflow and Control Hazards in RISC-V Architecture"** is a bonafide record of the work done by Group No. 4:

Adharsh K (Register No: NSS24CS010)
Angelino Saul (Register No: NSS24CS036)
Aswin Manoj (Register No: NSS24CS054)
Aswin R (Register No: NSS24CS055)

under our guidance during the academic year 2025–2026.

Signature of Guide
Name: Reshma S
Date:

Signature of HOD
Name: Anuraj Mohan
Date:

Acknowledgement

We express our sincere gratitude to **Reshma S**, our project guide, for their valuable guidance and support throughout this project.

We thank the Department of Computer Science and Engineering, NSS College of Engineering, Palakkad, for providing access to the necessary tools and resources.

We also acknowledge the open-source Ripes simulator project for enabling practical RISC-V pipeline analysis.

Abstract

This project analyzes the impact of buffer overflow vulnerabilities on the RISC-V processor pipeline. Utilizing the Ripes simulator, the study models a 5-stage RISC-V pipeline executing a custom RISC-V assembly program designed with a vulnerable function. The objective is to bridge the gap between theoretical memory mapping and practical instruction-level execution.

By intentionally overwriting the saved return address (`ra`) on the stack, the experiment demonstrates how a memory corruption vulnerability can escalate into a control flow subversion. The findings show that an overflow at `sp+12` corrupts the return address from its legitimate value (`0x00000004`) to a target address (`0x00000034`), successfully hijacking the program counter (PC) to execute a hidden code segment. Furthermore, the analysis reveals that this attack incurs a 2-cycle control hazard penalty as the processor pipeline resolves the `jr ra` instruction during the Execution (EX) stage.

Keywords: Buffer Overflow, RISC-V, Stack-Based Attack, Return Address, Control Hazard, Pipeline Flush, Ripes Simulator

Contents

Acknowledgement **3**

Abstract **4**

List of Figures **7**

List of Tables **8**

1 Introduction **9**
1.1 Background and Motivation 9
1.2 Problem Statement 9
1.3 Objectives 9
1.4 Study Scope and Constraints 10
1.5 Report Structure 10

2 Theoretical Background **11**
2.1 Memory and Stack Architecture in RISC-V 11
2.2 The 5-Stage Pipeline 11
2.3 Pipeline Hazards 11
2.4 Control Hazards and Branch Flush Penalty 12
2.5 Buffer Overflow: Definition and Classification 12
2.6 Stack-Based Buffer Overflow Mechanics 13
2.7 Return Address Corruption and PC Hijack 13

3 Methodology **14**
3.1 Experimental Setup (Ripes Configuration) 14
3.2 Program Design and Logic 14
3.3 Stack Memory Map 16
3.4 Instruction Flow Design 16
3.5 Measurement Procedure 16

4 Results and Observations **18**
4.1 Normal Stack Setup 18

4.2 Buffer Write Phase 18
4.3 The Overflow Event 19
4.4 Return Address Corruption 19
4.5 PC Hijack via `jr ra` 20
4.6 Control Hazard in the Pipeline 20
4.7 Memory Comparison (Before vs After) 20
4.8 Output Analysis Table 20

5 Discussion **22**
5.1 Memory Integrity Violation 22
5.2 Control Flow Hijacking 22
5.3 Pipeline Impact 22
5.4 Security Implications 23
5.5 Mitigations 23
5.6 Limitations of the Study 23

6 Conclusion **24**

A RISC-V Assembly Source Code **25**

List of Figures

3.1 Ripes simulator with 5-stage processor and program loaded. 14

4.1 Register file showing `ra=0x00000004` after legitimate save. 18
4.2 `sw t0,12(sp)` in MEM stage — overflow instruction executing. . . 19
4.3 `ra` corrupted to `0x00000034`; `li t1,999` is the current instruction. 19
4.4 `jr ra` in EX stage; `secret` instructions in ID and IF. 20
4.5 Side-by-side memory comparison showing return address corruption at sp+12. 21

List of Tables

2.1 Example Stack Frame Mapping 11
2.2 The 5 Stages of the Processor Pipeline 12
2.3 Types of Pipeline Hazards 12
2.4 Pipeline Trace for a Control Hazard on `jr ra` 12

3.1 Memory Layout within the `vulnerable` Function 16

4.1 Output Analysis: Register and Memory State at Each Execution Stage . 21

5.1 Mitigation Strategies and Effectiveness 23

Chapter 1

Introduction

1.1 Background and Motivation

Modern processors rely heavily on pipelining to increase instruction throughput and improve overall system performance. However, security vulnerabilities at the instruction level can uniquely affect architectural behavior. A classic example is the buffer overflow, which remains a historical and ongoing real-world threat to system integrity. By deeply analyzing how a buffer overflow manipulates processor state, we can understand the connection between high-level vulnerabilities and hardware-level execution. The Ripes simulator provides an invaluable platform for this, allowing architectural-level observation that is not possible in high-level languages like C or Python.

1.2 Problem Statement

"The core problem is to bridge the gap between theoretical concepts of memory mapping, instruction execution, and pipeline hazards with a practical demonstration of how vulnerabilities at the instruction level can propagate through processor stages and affect overall system behavior."

1.3 Objectives

The key objectives of this study are:

- To practically demonstrate a buffer overflow vulnerability at the instruction level using RISC-V assembly.
- To observe how a corrupted return address hijacks the processor's control flow.
- To trace the resulting control hazard through a 5-stage instruction pipeline.
- To quantify the pipeline penalty incurred by the illicit branch execution.

1.4 Study Scope and Constraints

This study is conducted under specific constraints to isolate the architectural effects:

- **Simulator:** Ripes v2.x, configured with a 5-stage pipelined processor.
- **ISA:** RV32IM (32-bit RISC-V integer and multiplication instruction set).
- **Attack Type:** Stack-based return address overwrite.
- **Environment Context:** The simulation runs bare-metal with no Operating System (OS) or memory cache modelling.
- **Nature of Exploit:** The overflow is controlled and intentionally induced via assembly logic, not through external user input or accidental bounds violation.

1.5 Report Structure

The rest of this report is organized as follows: Chapter 2 discusses the theoretical background, including the RISC-V stack architecture, pipelining, and buffer overflow mechanics. Chapter 3 details the methodology and experimental setup. Chapter 4 presents the results observed in the Ripes simulator. Chapter 5 discusses the implications and potential mitigations, followed by the conclusion in Chapter 6.

Chapter 2

Theoretical Background

2.1 Memory and Stack Architecture in RISC-V

In the RISC-V architecture, the stack traditionally grows downwards, meaning the stack pointer (`sp`) decreases as memory is allocated. A typical stack frame layout consists of local buffers, padding, and the saved return address (`ra`). For a frame containing a 2-word buffer and the saved return address:

- `sp+0` = `buf[0]`
- `sp+4` = `buf[1]`
- `sp+12` = `saved_ra`

Table 2.1: Example Stack Frame Mapping

Offset	Address (example)	Contents
`sp+0`	`0x7fffffe0`	`buf[0] = 10`
`sp+4`	`0x7fffffe4`	`buf[1] = 10`
`sp+8`	`0x7fffffe8`	(unused/padding)
`sp+12`	`0x7fffffec`	saved `ra` (TARGET)

2.2 The 5-Stage Pipeline

Pipelining allows multiple instructions to be processed simultaneously. The standard 5-stage RISC-V pipeline consists of the following stages:

2.3 Pipeline Hazards

Hazards prevent the next instruction in the instruction stream from executing during its designated clock cycle. There are three primary types of pipeline hazards:

Table 2.2: The 5 Stages of the Processor Pipeline

Stage	Name	Description
IF	Instruction Fetch	Fetches the instruction from memory using the PC.
ID	Instruction Decode	Decodes the instruction and reads registers.
EX	Execution	ALU performs arithmetic or resolves branch conditions/targets.
MEM	Memory Access	Reads or writes data to/from data memory.
WB	Write Back	Writes the result back to the register file.

Table 2.3: Types of Pipeline Hazards

Hazard Type	Description
Structural Hazard	Hardware cannot support the combination of instructions needed in the same clock cycle.
Data Hazard	Instruction depends on the results of a previous instruction still in the pipeline.
Control Hazard	Caused by a delay in determining the proper target address of a branch or jump.

2.4 Control Hazards and Branch Flush Penalty

Control hazards occur when the pipeline makes the wrong decision on branch prediction and fetches instructions that shouldn't be executed. In the tested RISC-V pipeline, branches resolve at the Execution (EX) stage. If a branch is taken, the instructions already loaded into the IF and ID stages must be flushed, resulting in a **2-cycle penalty** (Formula: Penalty = 2 cycles per taken branch).

Table 2.4: Pipeline Trace for a Control Hazard on `jr ra`

Instruction	C1	C2	C3	C4	C5
`jr ra`	IF	ID	EX	MEM	WB
Fetch+1		IF	flush		
Fetch+2			IF	flush	
`secret[0]`				IF	ID

2.5 Buffer Overflow: Definition and Classification

According to NIST, a buffer overflow occurs when a program attempts to put more data in a memory buffer than it can hold, leading to memory corruption. Vulnerabilities are generally classified into heap-based and stack-based overflows. In our study, we focus on a stack-based buffer overflow, specifically targeting `sp+12`, where the architecture stores the return address (`ra`) for function calls.

2.6 Stack-Based Buffer Overflow Mechanics

During normal execution, data is written exclusively to the allocated space (e.g., `sp+0` and `sp+4`). A stack-based overflow occurs when a write operation bypasses these bounds, writing to adjacent memory addresses. When an attacker intentionally writes to `sp+12`, they overwrite the saved `ra`, effectively changing the control return path.

2.7 Return Address Corruption and PC Hijack

In RISC-V, the return address registers (`x1` or `ra`) hold the address to return to after a subroutine completes. A subroutine call (`jal`) writes `PC+4` to `ra`. If the stack memory corresponding to the saved `ra` is overwritten, the `lw ra, 12(sp)` instruction during the function epilogue will load a corrupted value. Consequently, when the `jr ra` instruction executes, the Program Counter (PC) jumps to the illicit address, effectively hijacking control flow of the processor.

Chapter 3

Methodology

3.1 Experimental Setup (Ripes Configuration)

The experiment utilizes the Ripes simulator to visualize instruction execution and pipeline flow. Ripes is configured as follows:

- **Processor:** 5-Stage Pipelined
- **ISA:** RV32IM
- **Data Forwarding:** Default enabled
- **Branch Prediction:** None

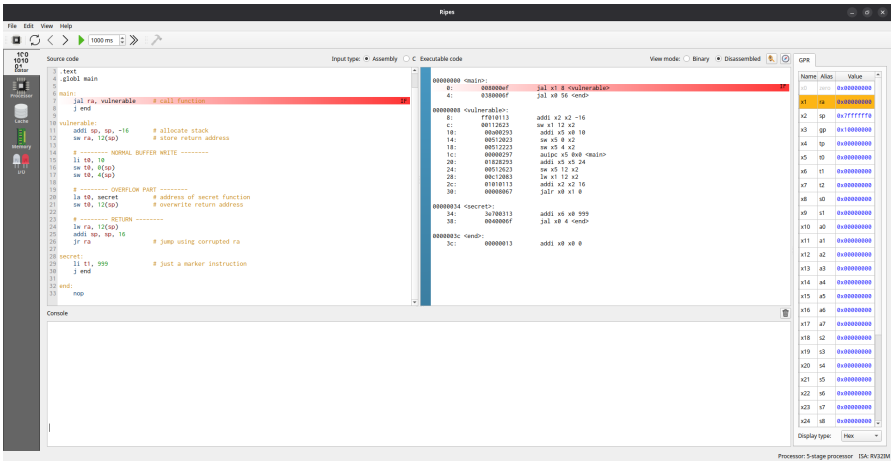


Figure 3.1: Ripes simulator with 5-stage processor and program loaded.

3.2 Program Design and Logic

A specific RISC-V assembly program was crafted to model the attack. It contains a `main` routine, a `vulnerable` function which allocates a stack frame, and a hidden `secret` function. The complete annotated code represents the target application:

```
.text
.globl main

main:
    jal ra, vulnerable      # call function; ra = addr of "j end"
    j end

vulnerable:
    addi sp, sp, -16        # allocate 16-byte stack frame
    sw ra, 12(sp)           # save legitimate return address

    # Normal buffer writes
    li t0, 10
    sw t0, 0(sp)            # buf[0] = 10
    sw t0, 4(sp)            # buf[1] = 10

    # Overflow: overwrite saved return address
    la t0, secret
    sw t0, 12(sp)           # OVERFLOW: saved ra -> address of secret

    # Return using corrupted ra
    lw ra, 12(sp)           # ra = address of secret (corrupted)
    addi sp, sp, 16
    jr ra                   # PC hijacked -> jumps to secret

secret:
    li t1, 999              # marker: proves execution reached here
    j end

end:
    nop
```

Listing 3.1: Vulnerable RISC-V Assembly Program

3.3 Stack Memory Map

The program allocates 16 bytes for the stack frame. The logical mapping is presented below, illustrating how the 12-byte offset creates a clear path to the return address:

Table 3.1: Memory Layout within the `vulnerable` Function

Offset	Address (Absolute)	Data Purpose
`sp+0`	`0x7fffffe0`	Buffer data
`sp+4`	`0x7fffffe4`	Buffer data
`sp+8`	`0x7fffffe8`	Unused
`sp+12`	`0x7fffffec`	Saved Return Address

3.4 Instruction Flow Design

The logical progression of the exploit follows six core phases based on instruction execution:

1. Function setup via `jal`.
2. Stack frame allocation and valid return address anchoring via `sw ra, 12(sp)`.
3. Standard buffer manipulation via localized stores.
4. The exploit trigger: a `sw` instruction explicitly targeting `12(sp)` with the address of `secret`.
5. Corrupted context restoration via `lw ra, 12(sp)`.
6. Control subversion via `jr ra`, violating intended control flow.

3.5 Measurement Procedure

The procedural steps for running the test are:

1. Load the compiled program into the Ripes simulator.
2. Single-step the execution to monitor precise architectural state changes.
3. Monitor the register panel for the value of `x1` (`ra`).
4. Monitor the memory panel for the hex values at the dynamic stack address coordinates.

5. Examine the pipeline diagram specifically during the execution of jump/branch instructions to identify flush conditions.

6. Capture the architectural state at each critical instruction to record empirical evidence of the attack chain.

Chapter 4

Results and Observations

4.1 Normal Stack Setup

Following the `jal` instruction into the `vulnerable` function, a 16-byte stack frame is allocated. The `sw ra, 12(sp)` instruction executes normally. In the register panel, `ra` (`x1`) holds the legitimate return value `0x00000004`. Correspondingly, the memory at `sp+12` is anchored precisely at `0x00000004`. At this point, the pipeline displays all five stages operating under normal IF/ID/EX/MEM/WB sequences.

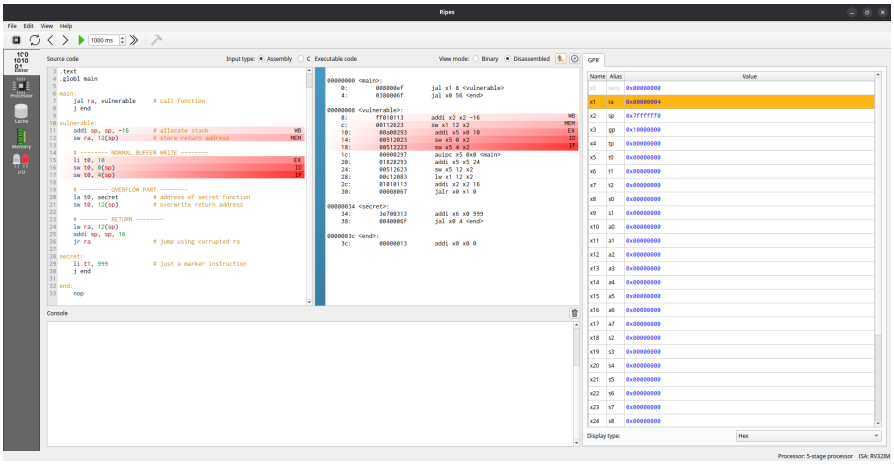


Figure 4.1: Register file showing `ra=0x00000004` after legitimate save.

4.2 Buffer Write Phase

The standard buffer writes execute next: `sw t0, 0(sp)` and `sw t0, 4(sp)`. Memory observations confirm that `sp+0` and `sp+4` now contain `0x0000000A` (the value 10). Crucially, the target slot at `sp+12` remains intact with the value `0x00000004`.

4.3 The Overflow Event

The critical simulated breach occurs at the execution of `sw t0, 12(sp)`. The instruction proceeds to the MEM stage where it overwrites the architectural return slot.

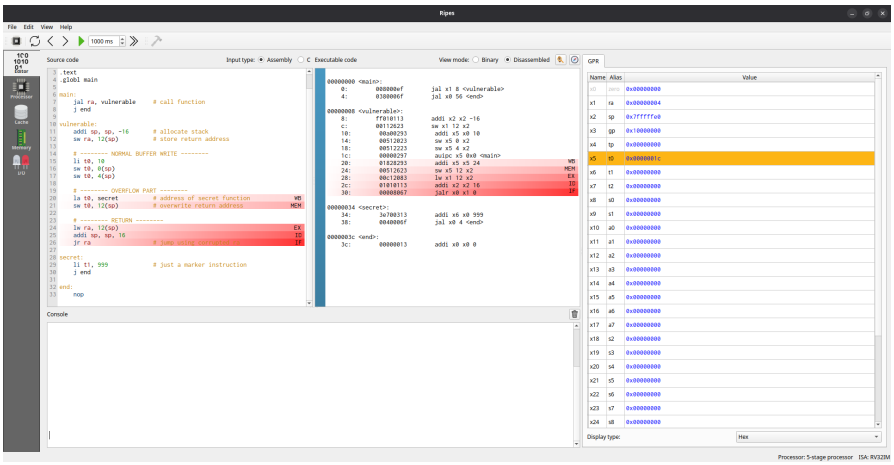


Figure 4.2: `sw t0,12(sp)` in MEM stage — overflow instruction executing.

Memory observation immediately highlights that `sp+12` has changed from `0x00000004` to `0x00000034` (the address of our `secret` function). This constitutes the core buffer overflow: a protected memory location is destructively overwritten.

4.4 Return Address Corruption

During the function epilogue, the `lw ra, 12(sp)` instruction restores the context. However, because memory was corrupted, the register file reads the overwritten value. The state of `ra` (`x1`) shifts from `0x00000004` to `0x00000034`.

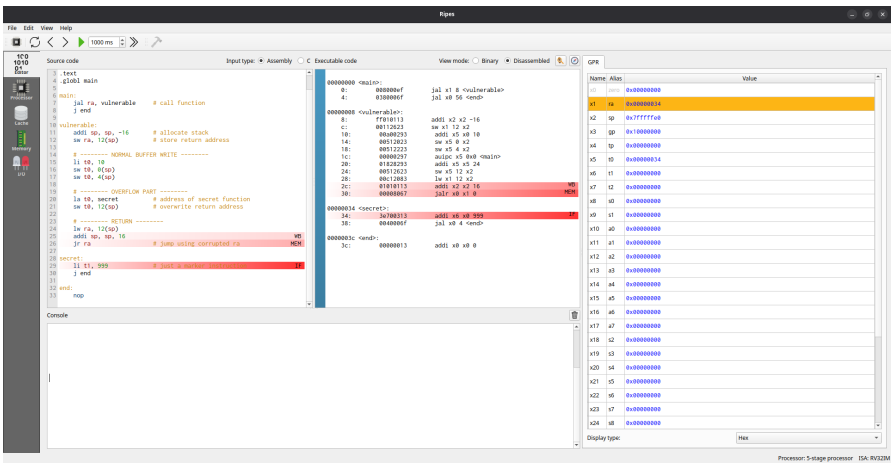


Figure 4.3: `ra` corrupted to `0x00000034`; `li t1,999` is the current instruction.

4.5 PC Hijack via `jr ra`

The consequence of the corrupted return address manifests via the `jr ra` instruction. Instead of returning control to `main`, the Program Counter explicitly jumps to `0x00000034`. Execution of `li t1, 999` operates as empirical proof that code inside the `secret` function is currently executing. The normal intended path (`main` → `vulnerable` → `main`) is transformed into the hijacked path (`main` → `vulnerable` → `secret`).

4.6 Control Hazard in the Pipeline

Monitoring the Ripes pipeline diagram exposes a critical effect on dynamic scheduling. The `jr ra` instruction resolves in the EX stage. Due to the branch, speculative fetches—such as `li t1, 999` in ID, and `j end` in IF—are fed into the pipeline. This introduces a control hazard where the processor cannot determine the target natively until EX finishes. Consequently, a 2-cycle penalty is incurred as these instructions represent the inadvertently accepted path.

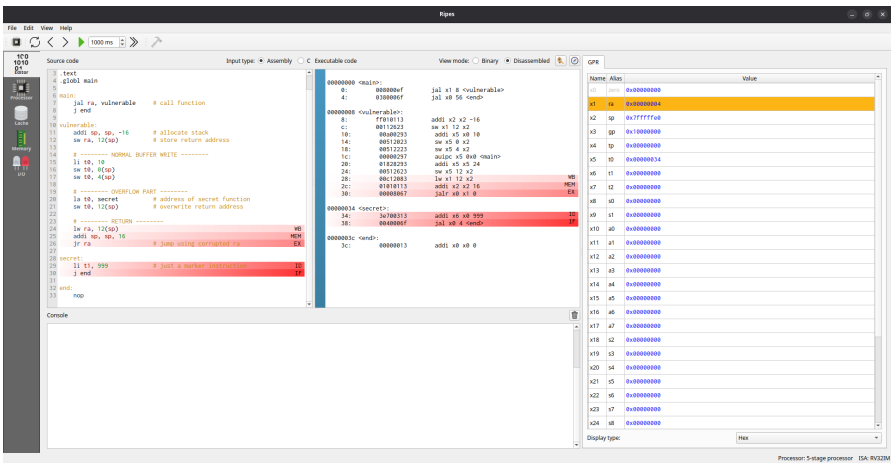


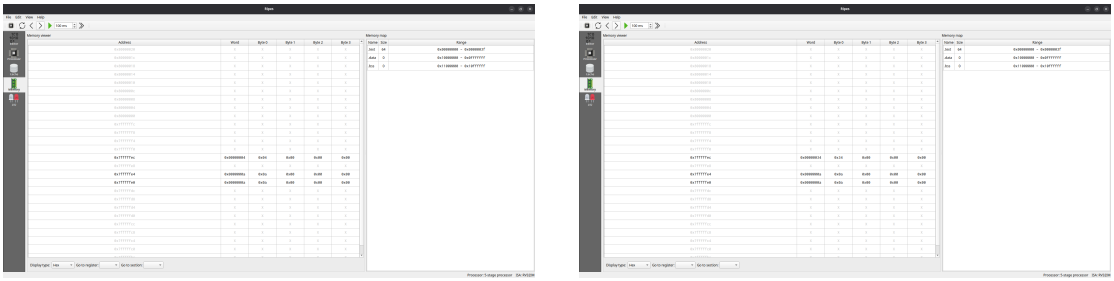
Figure 4.4: `jr ra` in EX stage; `secret` instructions in ID and IF.

4.7 Memory Comparison (Before vs After)

The pre-condition and post-condition memory dumps reinforce the precision of the attack vector.

4.8 Output Analysis Table

The empirical mapping of execution states over the entirety of the attack chain is compiled below:



Before: `sp+12 = 0x00000004` After: `sp+12 = 0x00000034`

Figure 4.5: Side-by-side memory comparison showing return address corruption at sp+12.

Table 4.1: Output Analysis: Register and Memory State at Each Execution Stage

Stage	Condition	Register/Memory State	Behavior
1	Before Function Call	`ra` = return to `main`	Normal execution begins
2	After `jal`	`ra` updated	Control to `vulnerable`
3	Stack Setup	`sp` decreased, `ra` at `12(sp)`	Stack frame created
4	Normal Buffer Write	`0(sp)`, `4(sp)` updated	Valid memory ops
5	Before Overflow	`ra` in memory = original	System stable
6	**After Overflow**	**`12(sp) = secret addr`**	**Return addr CORRUPTED**
7	Load Return Address	`ra` = address of `secret`	Corrupted value loaded
8	Function Return (`jr ra`)	`PC` = `secret`	Control flow changed
9	Execution at `secret`	`t1 = 999`	Unexpected instruction
10	Program End	Jump to `end`	Abnormal termination

Chapter 5

Discussion

5.1 Memory Integrity Violation

In standard architectural models like RISC-V, `sp+12` serves as a protected slot exclusively designed to hold the saved return address. However, from a hardware standpoint, this data is stored in unprivileged memory. As demonstrated, there is no hardware-level segregation preventing an instruction from overwriting structural metadata. The absolute absence of runtime bounds checking represents the root vulnerability in memory corruption exploits.

5.2 Control Flow Hijacking

Control flow hijacking is defined by the unauthorized redirection of a program's execution without an explicit logic call. The test proved that by precisely targeting the stack frame and replacing `ra` from `0x00000004` to `0x00000034`, the intended architectural return mechanism becomes an arbitrary execution jump. This reflects real-world techniques such as Return-Oriented Programming (ROP) attacks.

5.3 Pipeline Impact

The manipulation of an indirect jump (`jr ra`) introduces deliberate architectural strain. Because the branch address relies on an architectural register loaded from memory, it cannot resolve during IF or ID phases. Resolution at the EX stage strictly generates a 2-cycle control hazard penalty. Specifically, the processor speculatively fetched instructions from the illicit `secret` path, revealing that the pipeline inherently trusts corrupted registers until verification occurs.

5.4 Security Implications

Stack smashing and buffer overflows have dominated CVEs since the Morris Worm in 1988. While modern systems deploy complex layers of abstraction, this experiment proves that vulnerability fundamentally operates at the processor's most basic instruction level. RISC-V's simplified ISA clarifies this vulnerability chain: a single unrestricted store word (`sw`) instruction is sufficient to entirely subvert processor control flow.

5.5 Mitigations

Counteracting buffer overflows necessitates defense mechanisms acting on differing levels.

Table 5.1: Mitigation Strategies and Effectiveness

Mitigation	**Mechanism**	**Would it Stop This?**
Stack Canary	Inserts a sentinel value between the buffer and `saved ra`; validated before return.	Yes
ASLR	Randomizes memory layout so attacker cannot know address of `secret`.	Yes
DEP / NX	Marks the stack as non-executable.	Partial
Bounds Checking	Hardware or compiler explicitly restricts `sw` operations to allocated boundaries.	Yes
Safe Languages	Compiler natively prevents direct memory manipulation.	Yes

5.6 Limitations of the Study

This simulation establishes the mechanical baseline of a buffer overflow under intentionally restricted conditions. The overflow evaluated was manually crafted by direct assembly instructions rather than introduced via a malicious data payload triggering a logic flaw. Furthermore, the simulation environment intentionally omits operating system privilege rings, no execution (NX) bit implementations, hardware features like superscalar execution or deep out-of-order scheduling, and complex memory hierarchies such as separate instruction and data caches. The static address mapping in Ripes also simulates an environment entirely devoid of Address Space Layout Randomization (ASLR).

Chapter 6

Conclusion

This project successfully demonstrated the fundamental mechanics of a buffer overflow vulnerability using the RISC-V architecture. By constructing a targeted assembly program within the Ripes simulator, we observed the complete exploit chain: from the initial unauthorized `sw` instruction, through memory corruption, `ra` overwrite, and ultimately, `PC` hijack.

Visual verification at each pipeline stage conclusively confirmed that corrupting the return architectural slot effectively overrides processor control flow. When the illicit `jr ra` instruction executed, the pipeline encountered an indirect branch resolution delay, causing a measurable 2-cycle control hazard penalty as it speculatively processed unintended paths. This study bridges the theoretical understanding of control hazards with practical cybersecurity exploits, underlining the critical reality that processor architectures inherently execute instructions based on register and memory states without contextual validation. Consequently, advanced mitigation mechanisms involving both software compilation checks and hardware execution restrictions remain strictly necessary to ensure systematic memory integrity.

Chapter A

RISC-V Assembly Source Code

Below is the full, annotated RISC-V assembly source code used for the buffer overflow demonstration in the Ripes simulator. As executed, the function bypasses natural termination constraints through a stack memory violation.

```
.text
.globl main

main:
    jal ra, vulnerable      # call function; ra = addr of "j end"
    j end                   # Intended return point

vulnerable:
    addi sp, sp, -16        # allocate 16-byte stack frame
    sw ra, 12(sp)           # save legitimate return address

    # --- Normal execution boundaries ---
    li t0, 10
    sw t0, 0(sp)            # buf[0] = 10
    sw t0, 4(sp)            # buf[1] = 10

    # --- Exploit Trigger ---
    # Overflow: overwrite saved return address
    la t0, secret
    sw t0, 12(sp)           # OVERFLOW: saved ra -> address of secret

    # --- Function Epilogue ---
    # Return using corrupted ra
    lw ra, 12(sp)           # ra = address of secret (corrupted)
```

```
    addi sp, sp, 16
    jr ra                   # PC hijacked -> jumps to secret

secret:
    # --- Unauthorized Execution Zone ---
    li t1, 999              # marker: proves execution reached here
    j end

end:
    nop
```

Listing A.1: Complete Buffer Overflow Exploit Program